

Code for practical 2

```
object Practical2 {  
  def main(args: Array[String]): Unit = {  
  
    println("Shreyash Kadam SYCS")  
  
    val numbers = List(2, 4, 6, 8, 10, 6, 4, 2, 6, 8)  
  
    println(s"Numbers: $numbers")  
  
    val mean = numbers.sum.toDouble / numbers.length  
  
    val sorted = numbers.sorted  
  
    val median =  
      if (sorted.length % 2 == 0)  
        (sorted(sorted.length / 2 - 1) + sorted(sorted.length / 2)).toDouble / 2  
      else  
        sorted(sorted.length / 2).toDouble  
  
    var mode = numbers(0)  
    var maxCount = 0  
  
    for (i <- numbers) {  
      var count = 0  
  
      for (j <- numbers) {  
        if (i == j)  
          count += 1  
      }  
  
      if (count > maxCount) {  
        maxCount = count  
        mode = i  
      }  
    }  
  
    println(f"Mean: $mean%.2f")  
    println(f"Median: $median%.2f")  
    println(s"Mode: $mode")  
  }  
}
```

```
}
```

Output for practical 2

```
Shreyash Kadam SYCS
Numbers: List(2, 4, 6, 8, 10, 6, 4, 2, 6, 8)
Mean: 5.60
Median: 6.00
Mode: 6

Process finished with exit code 0
```

Code for practical 3

```
import scala.util.Random
import breeze.linalg._
```

```
object Practical3 {
  def main(args: Array[String]): Unit = {

    println("Shreyash Kadam SYCS")

    val data = DenseVector.fill(10)((Random.nextInt(10) + 1).toDouble)
    println(s"Random Dataset : $data")

    val mean = sum(data) / data.length

    def subtractMean(x: Double): Double = {
      x - mean
    }

    val difference = data.map(subtractMean)

    def square(x: Double): Double = {
      x * x
    }

    val squared = difference.map(square)

    val variance = sum(squared) / data.length
```

```

val stdDeviation = math.sqrt(variance)

println(f"Mean : $mean%.2f")
println(f"Variance : $variance%.2f")
println(f"Standared Deviation : $stdDeviation%.2f")
}
}

```

Output For practical 3

```

"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Shreyash Kadam SYCS
Random Dataset : DenseVector(7.0, 1.0, 2.0, 8.0, 7.0, 5.0, 7.0, 5.0, 9.0, 1.0)
Mean : 5.20
Variance : 7.76
Standared Deviation : 2.79

Process finished with exit code 0

```

Code for practical 4

```

import breeze.linalg._

object practical4 {
  def main(args: Array[String]): Unit = {
    println("Shreyash Kadam SYCS")

    val S1 = DenseVector(1.0, 2.0, 3.0, 4.0)
    val S2 = DenseVector(5.0, 6.0, 7.0, 8.0)

    println(s"Vector 1 : $S1")
    println(s"Vector 2 : $S2")

    val sum = breeze.linalg.sum(S1)
    println(f"Sum of Dense Vector S1 is : $sum%.2f")

    val mean = breeze.stats.mean(S2)
    println(f"Mean of Dense Vector S2 is : $mean%.2f")

    val Dotproduct = S1 dot S2
    println(f"Dot product of S1 and S2 Dense Vector is : $Dotproduct%.3f")

  }
}

```

```
}
```

Output for practical 4

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
Shreyash Kadam SYCS
Vector 1 : DenseVector(1.0, 2.0, 3.0, 4.0)
Vector 2 : DenseVector(5.0, 6.0, 7.0, 8.0)
Sum of Dense Vector S1 is : 10.00
Mean of Dense Vector S2 is : 6.50
Dot product of S1 and S2 Dense Vector is : 70.000
Process finished with exit code 0
```

Code for practical 5

```
import breeze.linalg._
import scala.util.Random

object practical5 {
  def main (args: Array[String]): Unit = {

    println("Shreyash Kadam SYCS")

    val row = 4
    val col = 4

    val S1 = Array.fill(row * col)(Random.nextInt(10) + 1)
    val matrix = new DenseMatrix( row, col, S1)

    println(s"Random Matrix : \n$matrix")

    val transpose = matrix.t
    println(s"transposed matrix : \n$transpose")

    val S4 = matrix.map(_._toDouble)
    val Determinant = det(S4)
    println(f"determinant : \n$Determinant%.2f")
  }
}
```

Output for practical 5

```
"C:\Program Files\Java\jdk-21.0.11\bin\java.exe" ...
```

```
Shreyash Kadam SYCS
```

```
Random Matrix :
```

```
7  8  10  7
```

```
1  7  4  3
```

```
5  4  9  4
```

```
8  3  6  9
```

```
transposed matrix :
```

```
7  1  5  8
```

```
8  7  4  3
```

```
transposed matrix :
```

```
7  1  5  8
```

```
8  7  4  3
```

```
10 4  9  6
```

```
7  3  4  9
```

```
determinant :
```

```
285.00
```

Code for practical Casestudy

```
import breeze.linalg.*
```

```
import org.apache.commons.math3.distribution.ChiSquaredDistribution
```

```
object casestudy {
```

```
  def main(args: Array[String]): Unit = {
```

```
    println("Shreyash Kadam SYCS")
```

```
    val points = List(
```

```
      120, 118, 115, 112, 110, 108, 105, 103, 101, 99,
```

```
      98, 96, 95, 94, 92, 90, 89, 88, 88, 88,
```

```
      87, 86, 85, 84, 83, 82, 81, 80, 79, 78,
```

```
      77, 76, 75, 74, 73, 72, 71, 70, 69, 68,
```

```
      67, 66, 65, 64, 63, 62, 61, 60, 59, 58,
```

```
      57, 56, 55, 54, 53, 52, 51, 50, 49, 48
```

```
    )
```

Shreyash Kadam

S091

```
println(s"\nTeam Points:")  
println(points)
```

```
val mean = points.sum.toDouble / points.length
```

```
val sorted = points.sorted
```

```
val median =  
  if (sorted.length % 2 == 0)  
    (sorted(sorted.length / 2 - 1) + sorted(sorted.length / 2)).toDouble / 2  
  else  
    sorted(sorted.length / 2).toDouble
```

```
var mode = points(0)  
var maxCount = 0
```

```
for (i <- points) {
```

```
  var count = 0
```

```
  for (j <- points) {
```

```
    if (i == j) {  
      count += 1  
    }
```

```
  }
```

```
  if (count > maxCount) {  
    maxCount = count  
    mode = i  
  }
```

```
}
```

```
val vector = DenseVector(points.map(_.toDouble).toArray)
```

```
val difference = vector.map(x => x - mean)
```

```
val squared = difference.map(x => x * x)
```

```
val variance = sum(squared) / vector.length
```

```
val standardDeviation = math.sqrt(variance)
```

```
println(f"\nMean      : $mean%.2f")
println(f"Median     : $median%.2f")
println(s"Mode       : $mode")
println(f"Variance    : $variance%.2f")
println(f"Standard Deviation : $standardDeviation%.2f")

}
}
```

Output for CaseStudy

```
Shreyash Kadam SYCS
Team Points:
List(120, 118, 115, 112, 110, 108, 105, 103, 101, 99, 98, 96, 95, 94, 92, 90, 89, 88, 88, 88, 87, 86, 85, 84, 83, 82, 81, 80, 79, 78, 77, 76, 75, 74, 73, 72, 71, 70, 69)

Mean      : 78.48
Median    : 77.50
Mode      : 88
```

```
Mean      : 78.48
Median    : 77.50
Mode      : 88
Variance  : 364.88
Standard Deviation : 19.10
```

```
Process finished with exit code 0
```